Computing (2020) 102:1447–1461
https://doi.org/10.1007/s00607-018-0681-y

Spatial-feature data cube for spatiotemporal remote sensing data processing and analysis

Dong Xu[1] · Yan Ma[2] · Jining Yan[3] · Peng Liu[2] · Lajiao Chen[2]

Received: 13 March 2018 / Accepted: 12 November 2018 / Published online: 20 December 2018
© Springer-Verlag GmbH Austria, part of Springer Nature 2018

Abstract

With the rapid development in Earth observation technology, a variety of satellite sensors have provided large and open sets of remote sensing data. However, traditional methods of analysis are no longer available for time-serial remote sensing data analysis that typically handles multidimensional spatio-temporal data models. Moreover, researchers have found it trivial and tedious to obtain ready-to-analyze data for Earth science models from regular Earth observation data. For an easy and efficient time-serial remote sensing data analysis, a spatial-featured data cube analysis tool based on multidimensional data model is proposed for time-serial remote sensing data processing and analysis. For the performance consideration, a distributed execution engine was also used for efficient implementation of large-scale tasks in parallel. Finally, through experiments on both normalized difference vegetation index product and water detection within a 20-year period, we confirmed that our approach is efficient and scalable for a long time-series analysis.

Keywords Spatial feature data cube · Long time series analysis · Multi-dimensional data · Remote sensing data processing

Mathematics Subject Classification 86

✉ Yan Ma
mayan@radi.ac.cn

✉ Jining Yan
yanjn@cug.edu.cn

[1] Institute of Remote Sensing and Digital Earth, University of Chinese Academy of Sciences, Beijing 100094, People's Republic of China

[2] Institute of Remote Sensing and Digital Earth, Chinese Academy of Sciences, Beijing 100094, People's Republic of China

[3] China University of Geosciences, Wuhan 430074, People's Republic of China

1 Introduction

In recent decades, the remarkable development in Earth observing (EO) technology provided a significant amount of remote sensing (RS) data openly available [11]. This large observation dataset characterized the information of earth surface in space, time, and spectral dimensions [24,25]. Apart form these dimensions, these data also contain many geographic features, such as forests, cities, lakes and so on, and these features could help researchers to locate their interested study areas rapidly. Now these multi-dimensional RS data with features have been widely used for global change detection research such as monitoring deforestation [13] and detecting temporal changes in floods [20]. However, the conventional geographic information system (GIS) tools are inconvenient for scientists to process the multidimensional RS data, because they lack appropriate methods to express multidimensional data model for analysis. And researchers have to do additional data organization work to conduct change detection analysis. For a more convenient analysis, they need a multidimensional data model which could support the seamless analysis in space, time, spectral and feature.

Recently, many researchers have proposed using multidimensional array model to organize the RS raster data [4,34]. Subsequently, they achieved the spatio-temporal aggregations capacity used in spatial on-line analytical processing (SOLAP) systems [1,10], as a data cube. Using this model, researchers can conveniently extract the desired data from the large dataset for analysis, and it reduced the burdens of data preparation for researchers in time-series analysis. However, in addition to extracting data with simple three-dimensional (3D) space-time coordinates, researchers occasionally need to extract data with some geographic features [6,29,30], which are often used to locate or mark the target regions of interest. For example, flood monitoring often need to process multidimensional RS data which own the characteristics of large covered range, long time series and multi bands [3]. If we built all the analysis data as a whole data cube which own the laker or river features, we could rapidly find the target study area we need by feature and analyse the multi bands image data to detect the flood situation with the time series. That make researchers could focus on their analysis work without being troubled by the data organization. This paper aims to develop the spatial-feature remote sensing data cube (SRSDC), a data cube whose goal is to deliver a spatial-feature-supported, efficient, and scalable multidimensional data analysis system to handle the large-scale RS data. The SRSDC provides spatial feature repositories to store and manage the vector feature data, and a feature translation to transform the spatial feature information to a query operation. To support large-scale data analysis, the SRSDC provides a works-scheduler architecture to process the sliced multidimensional arrays with dask [19].

The remainder of this paper is organized as follows. Section 2 describes the related works. Section 3 presents an architectural overview of the SRSDC. Section 4 presents the design and implementation of feature data cube and distributed execution engine in the SRSDC. Section 5 uses the long time-series remote sensing production process and analysis as examples to evaluate the performance of feature data cube and distributed execution engine. Section 6 concludes this paper and describes the future work prospects.

2 Related work

With the growing numbers of archived RS images for Earth observation, an increasing number of scientists are interested in the spatiotemporal analysis of RS data. Many researchers proposed combining online analytical processing (OLAP) [26] technology with the GIS [32] to build a data cube. They built the multidimensional database paradigm to manage several dimension tables, periodically extracting the dimension information from the data in GIS, and achieved the ability to explore spatiotemporal data using the OLAP space–time dimension aggregation operation. Sonia Rivest et al. [18] deployed a spatial data warehouse based on GIS and spatial database to acquire, visualize, and analyze the multidimensional RS data. Matthew Scotch et al. developed the SOVAT tool [22], using OLAP and GIS to perform the public health theme analysis with the data composed of spatiotemporal dimensions. These tools can facilitate researchers to extract data with spatiotemporal dimensions; however, their multidimensional data model is unsuitable for complicated scientific computing. Further, they did not adopt a appropriate architecture for large data processing [9,31]. Therefore, their ability to handle large-scale data is limited.

Owing to natural raster data structure of Earth observation images, the time-series imagery set can be easily transformed to multidimensional array. For example, a 3D array can represent the data with spatiotemporal dimensions. This data type is suitable for parallel processes, because a large array can be easily partitioned into several chunks for distributed storing and processing. In addition, the multidimensional array model enables a spatiotemporal auto-correlated data analysis; therefore, researchers need not be concerned about the organization of discrete image files. Thus, many researches are focused on developing new analysis tools to process the large RS data based on the multidimensional array model, e.g., Gamara et al. [5] tested the performance of spatiotemporal analysis algorithms on array database architectures—SCIDB [21], which described the efficiency of spatiotemporal analysis base on the multidimensional array model, Assis et al. [4] built a parallel RS data analysis system based on the MapReduce framework of Hadoop [2], describing the 3D array with key/value pairs. Although these tools have significantly improved the computation performance of RS data analysis, they also contain some deficiencies. First, many of them focused only on analyzing the RS raster image data located by geographic coordinates, and did not provide the support of spatial feature, thereby limiting their ability to use these geographic objects in the analysis application. Next, some of these tools require analysers to fit their algorithms into specialised environments, such as Hadoop MapReduce framework [23]. This will be user unfriendly to researchers who only desire to focus on their analysis application.

3 Architecture overview

3.1 Target data and environment

The SRSDC system is designed for providing the services of large RS data time-series analysis with spatial features, and it aims to manage and process the large-scale spatial

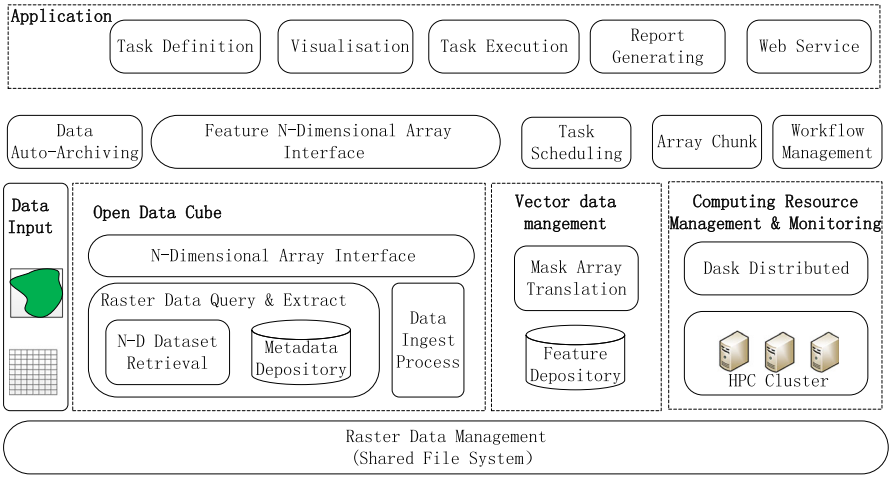


Fig. 1 The architecture of the SRSDC

feature data and satellite images seamlessly. Open data cube (ODC) [16] is a popular data cube system used for spatial raster data's management and analysis. Based on the ODC, we archived large amounts of satellite data within China. These data was come from different satellites including Landsat, MODIS, GaoFen (GF) and HuanJing (HJ). In addition, the SRSDC also contains many features data within China, such as lakes, forests and cities. These spatial vector data was downloaded from the official web site of OpenStreetMap [17]. Before obtaining these satellite images into the SRSDC, the geometric correction and radiometric correction for these images must be ensured. This can ensure the comparability between the images in different time, space and measurements; subsequently, the global subdivision grid can be used to partition the data into many tiles (grid files). These tiles were stored as NetCDF format [28], which supports many analysis libraries and scientific toolkits.

3.2 FRSDC architecture overview

The SRSDC system adopted the relational database and file system to manage the spatial data. It is designed to be scalable, efficient and feature support for time-series analysis. Compared with ODC system [16], which only support spatial raster data's management and analysis as a data cube, the SRSDC supports the extraction of target satellite data as a multidimensional array with the geographic object. It could perform the spatial query operation with geographic objects, instead of locating data with only geographic coordinates. Therefore, the dataset built for analysis owns the geographic meaning. Thus, if researchers desire to obtain the target dataset, they would only need to query data with geographic meaning of the analysis themes, without knowing specific geographic coordinates. As shown in Fig. 1, the system is primarily composed of the data management and distributed execution engine (DEE). Data management consists of two parts, raster data management and vector data management. For the raster data, the SRSDC will archive it into shared network file system and extract its metadata

information automatically; these metadata will be stored into the metadata depository managed by ODC. For the vector data, the SRSDC stores them as geographic objects in the spatial database. Upon the data management, an N-Dimensional array interface is responsible for transforming the raster data and vector data to an N-Dimensional array that owns the spatial feature information. Xarray [12] is used for array handing and computing. DEE is responsible for providing the computing environment and resource on high performance computing (HPC) clusters. The SRSDC use dask [19], which is a parallel computation library with blocked algorithms, for the task scheduling, distributed computing, and resource monitoring. It could help researchers to execute the analysis tasks in parallel.

4 Design and implementation

4.1 Feature data cube

Feature Data Cube is a spatial data management system which could manage both the feature data and image data as a data cube. And it could help researchers to finish the spatial analysis work more efficiently.

4.1.1 Spatial feature object in FRSDC

Spatial feature is a geographic object that owns the special geographic meaning. It is often important for RS application, because researchers occasionally need to process the RS image dataset with geographic objects, such as the classification of RS image with spatial features [7,14,15]. However, many RS data cube systems only provide the multidimensional dataset without features. Hence, researchers are required to perform additional work to prepare the data for analysis. For example, ODC system [16] and the data cube based on SCIDB [5] could only query and locate the study area by simple geographic coordinates, so researchers must transform their interested spatial features to coordinate ranges one by one if they want to prepare the analysis ready data. To solve this problem, the SRSDC combined the basic N-Dimensional array with geographic objects to provide the feature N-Dimensional array for researchers, and researchers could easily organise the analysis ready N-Dimensional dataset by their interested features. Within the SRSDC, now we primarily archive the forest and lake features of China, and stored them as geographic objects in PostGIS database. The unified modeling language (UML) class diagram in the SRSDC is shown in Fig. 2, and the description of these classes is as follows:

1. The feature class is provided for users to define their spatial feature of interest with a geographic object. It contains the feature type and the geographic object.
2. The feature type class represents the type of geographic object, such as lakes, forests, cities and so on. It contains the description of the feature type and analysis algorithm names that are suitable for the feature type.
3. The geographic object class describes the concrete vector data with geographic meaning, such as Poyang Lake (a lake in China). It contains the vector data type to illustrate its geometry type.

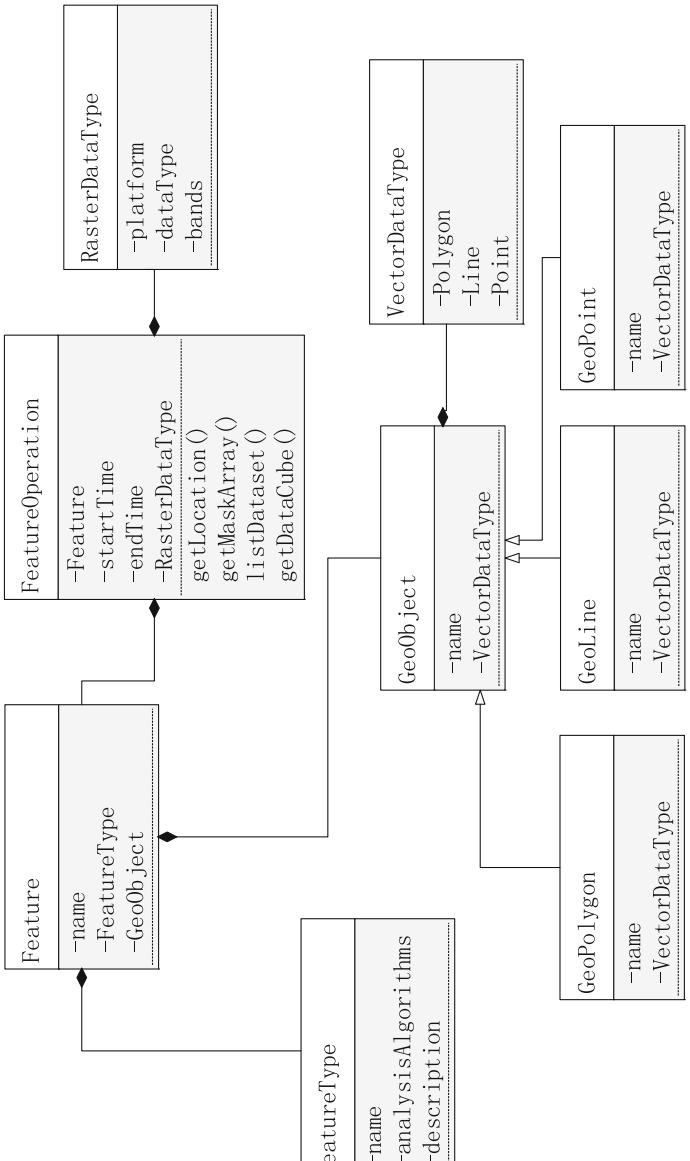


Fig. 2 The UML calss diagram in the SRSDC

4. The raster data type class is used to describe the type of satellite data. It contains the satellite platform, product type, and bands information.
5. The feature operation class is used to extract the feature datacube dataset from the SRSDC. It contains the feature object, raster data type, and time horizon to build the target feature N-Dimensional array. It also provide some operation functions for users.

4.1.2 Data management

Data management consists of raster data management and vector data management. Because of satellite data's large volume and variety, the SRSDC uses the file system to store the raster data and manage the metadata by a relation database that contains NoSQL fields. In the metadata depository, the SRSDC uses NoSQL fields to describe the metadata information instead of full relation model. This is because the number of satellite sensors is increasing rapidly, and if the full relation model is used, the database schema must be expanded frequently to meet the new data sources. In contrast, NoSQL fields are more flexible in describing the metadata of satellite data that originate from different data sources. The NOSQL fields contain the time, coordinate, band, data URL, data type, and projection. Among these fields, Some are used for data query, such as the time, band, coordinates, and data type. Some other fields are used for loading the data and building the multidimensional array in memory, such as data URL and projection. In addition, compared the vector data volume (GB level) we download from the OpenStreetMap [17] with the raster data volume (TB level) we archived from several satellite data centers [33], we found that most spatial features that are represented by the vector data are not as large as the raster images, therefore we established a feature depository instead of file system to store and manage them as geographic objects. These objects may contain different geographic meanings, and we defined them as feature types.

The runtime implementation of feature data cube building and processing is shown in Fig. 3. First, the SRSDC receives the user's data request from the web portal and obtains the target geographic object by querying the feature depository. Next, it conducts a feature translation to transform the geographic object into mask array and obtains the minimum bounding rectangle (MBR) of the feature. Subsequently, with the vertex coordinates of MBR and time horizon, the SRSDC searches for the required raster data's metadata to locate physical URLs of the raster data. Next, ODC's N-Dimensional array interface will load the raster data set from file system and build a multidimensional array in memory. Subsequently, the mask array will be applied to masking the multidimensional array, and a new multidimensional array with features for analyzing and processing will be obtained. Finally, the SRSDC will process the data with relevant algorithm and return the analysis results to the user.

4.2 Distributed executed engine

As an increasing number of RS applications needs to process or analyze the massive volume of RS data collections, the stand-alone mode processing can not satisfy the

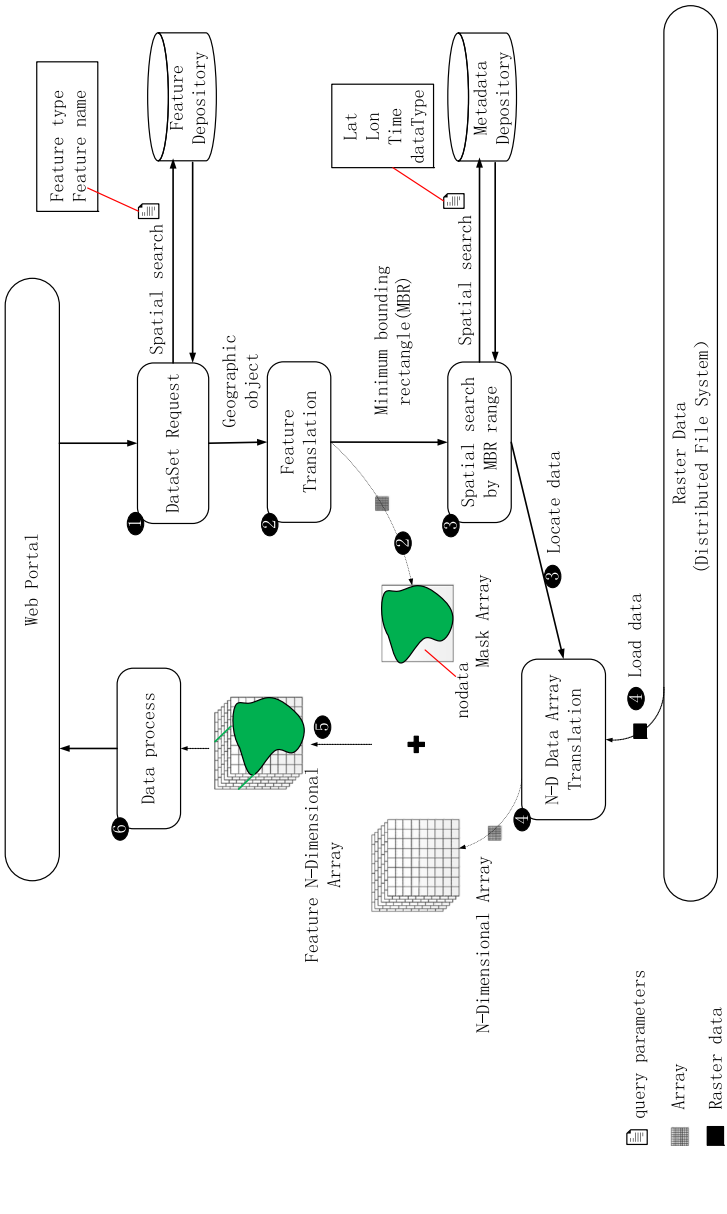


Fig. 3 Runtime implementation of feature data cube building

computation requirement. To process the large-scale RS data efficiently, we built a distributed executed engine using the dask—a distributed computing framework focusing on scientific data analysis. Compared with the popular distributed computing tools such as Apache Spark, dask supports the multidimensional data model natively and owns the similar API with pandas and numpy. Therefore, it is more suitable for the computing of N-Dimensional array. Similar to spark, dask is also a master-slave system framework that consists of one schedule node and several work nodes. The schedule node is responsible for scheduling the tasks, while the work nodes are responsible for executing tasks. If all the tasks have being performed, these workers' computation results would be reduced to the scheduler and the final result would be obtained.

In the SRSDC, we could index the satellite image scenes by adding their metadata information to database, and then obtain the data cube dataset (N-Dimensional arrays) from the memory for computing. However, to compute the large global dataset, we should slice the large array into the fixed-size sub-arrays called chunks for computing in the distributed environment. The SRSDC partitions these native images into seamless and massive tiles based on a latitude/longitude grid. The tile size is determined by the resolution of satellite images. For example, in the SRSDC, the Landsat data (each pixel 0.00025°) was partitioned into tiles of size $1° \times 1°$, and the tiles (4000×4000 pixels array) can be easily organized as data chunk, which is suitable for the memory in the worker node. By configuring the gird number and time horizon, the chunk could be built. Further, with these data chunks, the SRSDC can transform the big dataset (N-Dimensional arrays) to several sub-arrays loaded by different worker nodes. After all the data chunks have been organized, The scheduler will assign the chunks to the workers and map the functions for computing.

As shown in Fig. 4, the processing of large-scale time series analysis by the distributed executed engine is as follows:

1. Organize the data cube dataset by multidimensional spatial query.
2. Configure the appropriate parameters (grid number or time horizon) to organized the data chunks for workers, manage the chunks' ids with a queue.
3. Select the analysis algorithm and data chunks to compose the tasks, and assign these tasks to the worker node.
4. Check the executing state of each task in the workers, if failure occurs, recalculate the result.
5. Reduce all the results to the scheduler and return the analysis result to the client.

5 Experiments

To verify the ability of multidimensional data management and large-scale data analysis in the SRSDC, we conducted the following time-series analysis experiments focusing on spatial feature regions and compared the performance of GEE and standalone mode processes on the target dataset.

In this experiment, two RS application algorithms for time-series change detection have been used: normalized difference vegetation index (NDVI) for the vegetation change detection and water observation from space (WOfs) for the water change

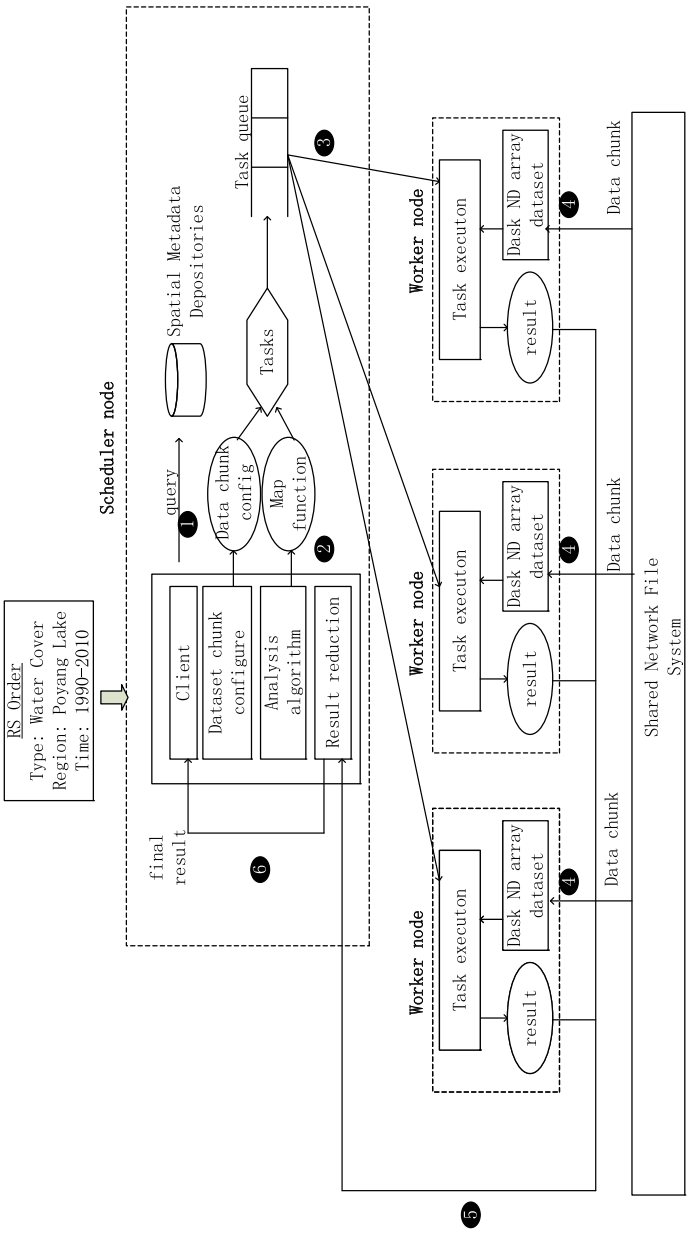


Fig. 4 Large-scale RS analysis processing with distributed executed engine

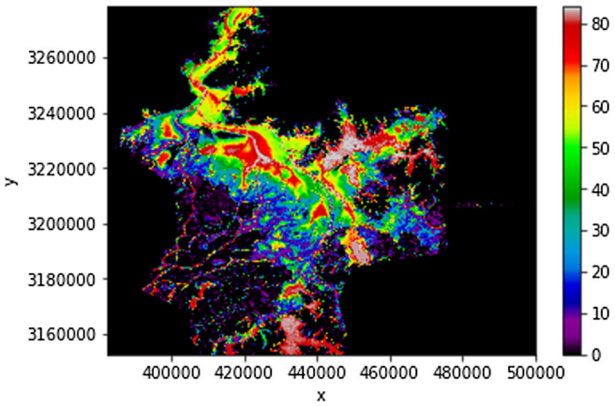


Fig. 5 Water area of Poyang Lake over 20-year time series

detection. We built the distributed executed engine with four nodes connected by a 20 GB Infiniband network; one node for the scheduler and tree nodes for the workers. Each node was configured with Inter(R) Xeon(R)E5-2460 CPU(2.0 GHz) and 32GB memory. The operating system is CentOS 6.5, and the python version is 3.6.

To test the performance of feature data cube, we selected two study regions with the special features as examples. One region for the NDVI is Mulan hunting ground, Hebei Province, China (40.7°–43.1°N, 115.8°–119.1°E), and another region for WOfs is Poyang Lake, Jiangxi Province, China. We build two feature data cube datasets for 20 years (1990–2009) with Landsat L2 data and the geographic objects. The data volume for Mulan hunting ground is approximately 138 GB and the data volume for Poyang Lake is about 96 GB. Figure 5 shows the percentage of observations detected as water for Poyang Lake over the 20-year time series. The red area represents the frequent or permanent water, and the purple area represent the infrequent water. From the result, the shape area of Poyang Lake can be observed clearly. Figure 6 shows the annual average NDVI productions on the Mulan hunting ground; Fig. 7 shows the NDVI time series result of the sampling site (41.5620°N, 117.4520°E) over 20 years. As shown, the values during 2007–2008 were abnormally below the average. This is because the average annual rainfall during this time is lower than that in normal years.

To test the processing performance of the DEE for different amount of data, we tested time consumed by processing 6.3 GB, 12.8 GB, 49.6 GB, 109.8 GB, 138.6 GB input data for NDVI production. These data has been partitioned into 4000 $\times$ 4000 pixels tiles mentioned above, with which we compared the performances of the stand-alone model and DEE models:

1. *Stand-alone model* Organize the dataset as data chunks, and process these data chunks serially with a single server.
2. *DEE model* Organize the large dataset as data chunks, and assign different workers to read these data chunks to process them in parallel with distributed executed engine, which consist of one schedule node and tree work nodes.

As shown from the experimental results in Fig. 8, the DEE mode is much faster than the stand-alone mode because it can use the shared memory of clusters nodes and

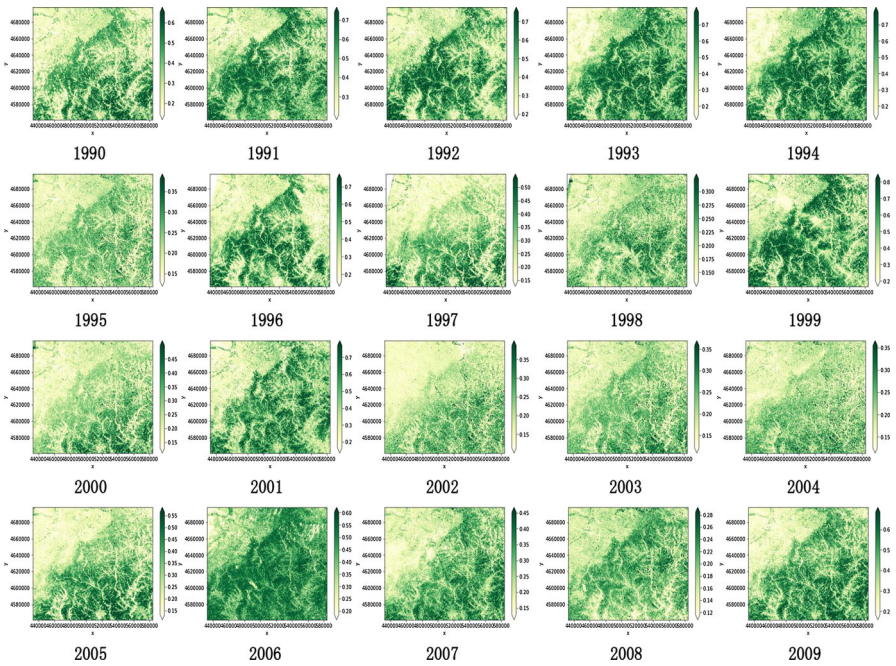


Fig. 6 The annual average NDVI of Mulan hunting ground for 20 years

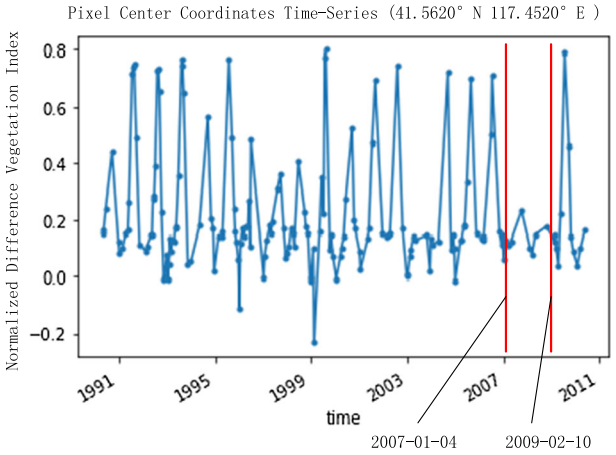


Fig. 7 A NDVI time series of sampling site on Mulan hunting ground

process the large dataset in parallel. As the process data amount increases, we also observed that the time consumed will grow nonlinearly. This is due to the IO limit of the shared network file system and scheduling overhead. The speedup performance when generating the NDVI production with increasing numbers of work nodes also proved this point, as shown in Fig. 9. Therefore, we conclude that the SRSDC has a certain capacity to process the massive data, which is unsuitable for the memory.

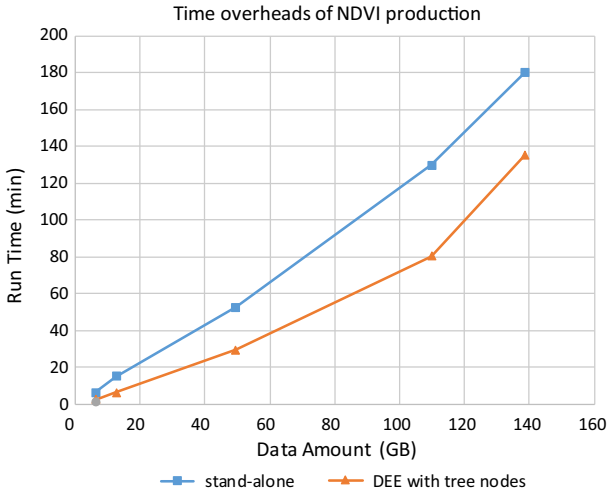


Fig. 8 Runtime of NDVI with the increase of data volume

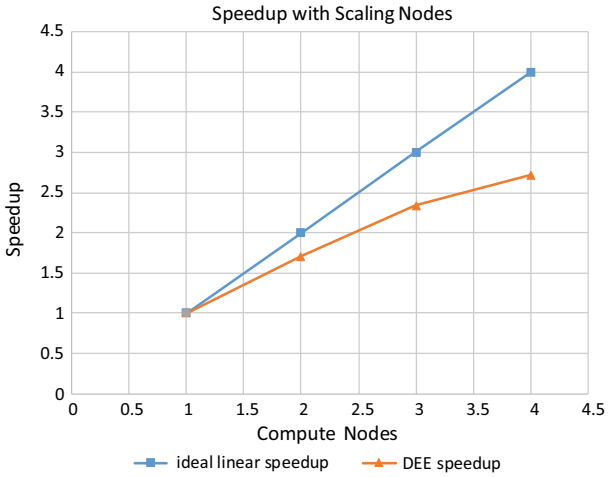


Fig. 9 Speedup for the generation of NDVI products with increasing nodes

6 Conclusion

We have designed and tested a feature supporting, scalable, and efficient data cube for time-series analysis application, and used the spatial feature data and remote sensing data for comparative study of the water cover and vegetation change. In this system, the feature data cube building and distributed executor engine are critical in supporting large spatiotemporal RS data analysis with spatial features. The feature translation ensures that the geographic object can be combined with satellite data to build a feature data cube for analysis. Constructing a distributed executed engine based on dask ensure that the efficient analysis of large-scale RS data. This work could provide

a convenient and efficient multidimensional data services for many remote sensing applications [8,27]. However, it also have some limitations, for example, the image data is stored in the shared file system, it's IO performance is limited by the network.

In the future, more work will be performed to optimize the system architecture of the SRSDC, such as improving the performance of distributed executed engine, selecting other storage methods which could ensure the process data locality, adding more remote sensing application algorithms, etc.

Acknowledgements This research was supported by the National Natural Science Foundation of China (No. 41401512), National Natural Science Foundation of China (No. 41471368), National Key Research and Development Plan of China (No. 2016YFA0600302), and Youth Innovation Promotion Association of the Chinese Academy of Sciences (No. Y6YR0300QM).

References

1. Ahmed TO (2008) Spatial on-line analytical processing (SOLAP): overview and current trends. In: 2008 international conference on advanced computer theory and engineering, pp 1095–1099
2. Apache (2017) Hadoop web site. http://hadoop.apache.org/. Accessed 05 Dec 2017
3. Asante KO, Macuacua RD, Artan GA et al (2007) Developing a flood monitoring system from remotely sensed data for the Limpopo basin. IEEE Trans Geosci Remote Sens 45(6):1709–1714. https://doi.org/10.1109/TGRS.2006.883147
4. Assis LF, Ribeiro G et al (2016) Big data streaming for remote sensing time series analytics using MapReduce. In: XVII Brazilian symposium on geoinformatics
5. Camara G, Assis LF et al (2016) Big earth observation data analytics: matching requirements to system architectures. In: Proceedings of the 5th ACM SIGSPATIAL international workshop on analytics for big geospatial data, BigSpatial '16. ACM, New York, NY, USA, pp 1–6
6. Chen W, Li X, He H et al (2017) Assessing different feature sets' effects on land cover classification in complex surface-mined landscapes by ZiYuan-3 satellite imagery. Remote Sens 10(2):2–10
7. Chen W, Li X, He H et al (2018) A review of fine-scale land use and land cover classification in open-pit mining areas by remote sensing techniques. Remote Sens 10(1):2–14
8. Chen C, Liu X, Qiu T et al (2017) Latency estimation based on traffic density for video streaming in the internet of vehicles. Comput Commun 111:176–186. https://doi.org/10.1016/j.comcom.2017.08.010. http://www.sciencedirect.com/science/article/pii/S014036641730275X. Accessed 27 Nov 2017
9. Fan J, Yan J, Ma Y et al (2018) Big data integration in remote sensing across a distributed metadata-based spatial infrastructure. Remote Sens 10(1):7
10. Gonzlez DB, Gonzlez LP (2013) Spatial data warehouses and SOLAP using open-source tools. In: 2013 XXXIX Latin American computing conference CLEI, pp 1–12
11. Guo H, Wang L, Chen F et al (2014) Scientific big data and digital earth. Chin Sci Bull 59(35):5066–5073
12. Hoyer S, Hamman JJ (2017) xarray: N-d labeled arrays and datasets in Python. J Open Res Softw 5(3):1–3
13. Kennedy RE, Yang Z, Cohen WB (2010) Detecting trends in forest disturbance and recovery using yearly landsat time series: 1. landtrendr temporal segmentation algorithms. Remote Sens Environ 114(12):2897–2910
14. Li X, Chen G, Liu J et al (2017a) Effects of rapideye imagery's red-edge band and vegetation indices on land cover classification in an arid region. Chin Geogr Sci 27(5):827–835
15. Li X, Chen W, Cheng X et al (2017b) Comparison and integration of feature reduction methods for land cover classification with rapideye imagery. Multimed Tools Appl 76(21):23041–23057
16. ODC (2017) Open data cube. http://datacube-core.readthedocs.io/en/latest/index.html. Accessed 13 Oct 2017
17. OpenStreetMap (2017) The project that creates and distributes free geographic data for the world. http://www.openstreetmap.org. Accessed 24 Oct 2017

18. Rivest S, Bdard Y, Proulx MJ, Nadeau M, Hubert F, Pastor J (2005) SOLAP technology: merging business intelligence with geospatial technology for interactive spatio-temporal exploration and analysis of data. ISPRS J Photogramm Remote Sens 60(1):17–33
19. Rocklin M (2015) Dask: parallel computation with blocked algorithms and task scheduling. In: Huff K, Bergstra J (eds) Proceedings of the 14th Python in science conference, pp 130–136
20. Sakamoto T, Nguyen NV, Kotera A et al (2007) Detecting temporal changes in the extent of annual flooding within the Cambodia and the Vietnamese Mekong Delta from MODIS time-series imagery. Remote Sens Environ 109(3):295–313
21. SCIDB (2017) A database management system designed for multidimensional data. http://scidb.sourceforge.net/project.html. Accessed 01 Nov 2017
22. Scotch M, Parmanto B (2005) SOVAT: spatial OLAP visualization and analysis tool. In: Proceedings of the Hawaii international conference on system sciences, p 142.2
23. Shvachko K, Kuang H, Radia S et al (2010) The hadoop distributed file system. In: IEEE symposium on MASS storage systems and technologies, pp 1–10
24. Song W, Wang L, Xiang Y et al (2017) Geographic spatiotemporal big data correlation analysis via the Hilbert–Huang transformation. J Comput Syst Sci 89:130–141
25. Song W, Wang L, Liu P et al (2018) Improved t-SNE based manifold dimensional reduction for remote sensing data processing. Multimed Tools Appl. https://doi.org/10.1007/s11042-018-5715-0
26. Thomsen E (2002) OLAP solutions: building multidimensional information systems. Wiley, Hoboken
27. Tian Y, Li X, Sangaiah AK et al (2018) Privacy-preserving scheme in social participatory sensing based on secure multi-party cooperation. Comput Commun 119:167–178. https://doi.org/10.1016/j.comcom.2017.10.007. http://www.sciencedirect.com/science/article/pii/S0140366417310599. Accessed 08 Feb 2018
28. UCAR (2017) NetCDF file format and API. http://www.unidata.ucar.edu/software/netcdf/
29. Wang L, Song W, Liu P (2016) Link the remote sensing big data to the image features via wavelet transformation. Clust Comput 19(2):793–810
30. Wang L, Zhang J, Liu P et al (2017) Spectral-spatial multi-feature-based deep learning for hyperspectral remote sensing image classification. Soft Comput 21(1):213–221
31. Yan J, Ma Y, Wang L et al (2017) A cloud-based remote sensing data production system. Future Gener Comput Syst 86:1154–1166. https://doi.org/10.1016/j.future.2017.02.044
32. Yijiang Z (2012) The conceptual design on spatial data cube. In: 2012 2nd international conference on consumer electronics, communications and networks (CECNet), pp 645–648
33. Zhang J, Yan J, Ma Y et al (2016) Infrastructures and services for remote sensing data production management across multiple satellite data centers. Clust Comput 19(3):1–18
34. Zhang L, Chen H, Sun X et al (2017) Designing spatial–temporal–spectral integrated storage structure of multi-dimensional remote sensing images. J Remote Sens 21(1):62–73

Computing is a copyright of Springer, 2020. All Rights Reserved.